

Claude Certified Architect — Foundations (CCA-F)

One-page cheat sheet · UNOFFICIAL community prep · verify against anthropic.com

EXAM SNAPSHOT

OFFICIAL: CCA-F = Anthropic's 1st Claude cert · for solution architects · on Anthropic Academy/Skilljar · access via free Partner Network

COMMUNITY (verify): ~60 scenario MCQs · 120 min · scaled 100–1000, pass ~720 · ~\$99/attempt · ~2-yr validity

Theme across all domains: **least complexity that works** · **enforce hard rules in code (hooks)** · **curate context** · **test yourself to study**

D1 · Agentic Architecture (~27%)

- Workflow = predefined code paths; Agent = LLM directs its own loop
- Patterns: chaining, routing, parallelization, orchestrator-workers, evaluator-optimizer
- Agentic loop: gather context > act (tools) > verify > repeat; always a STOP condition
- Default single-agent; multi-agent only for broad/parallel work (~15x tokens)
- #1 multi-agent failure: vague delegation; give subagents objective+scope+tools+format
- Workers return condensed summaries; persist state externally for long runs
- Hard requirement? Use a deterministic hook, not a

D3 · Tools & MCP (~18%)

- Loop: send tool defs > model emits tool_use > YOUR app runs it > return tool_result
- Your app executes tools (not the model) = the security boundary
- Good tool: clear name, rich description (=a prompt), typed schema, concise results
- MCP = open standard, 'USB-C for AI'. Host > Client (1:1) > Server
- 3 primitives: Tools (model), Resources (app), Prompts (user)
- Use MCP to write once, reuse across hosts; local=stdio, remote=HTTP/SSE
- Remote server security: require auth; validate/scope

D5 · Claude Code (~20%)

- Terminal agentic coding tool; also runs headless for CI/automation
- Config layers: enterprise/managed (top) > project (committed) > local (gitignored)
- CLAUDE.md = persistent auto-loaded memory; keep lean; NEVER secrets
- Hooks = deterministic lifecycle commands; PreToolUse can BLOCK; PostToolUse auto-lint
- Subagents: own context window + system prompt + restricted tools
- Skills = SKILL.md folder, loaded progressively when relevant
- Headless CI (no human): least privilege + hooks + logging are mandatory

D2 · Prompt Engineering (~20%)

- Ladder: clear+direct > examples > chain-of-thought > XML tags > role > prefill > chain
- Few-shot examples: relevant, diverse, delimited (avoid one-answer bias)
- Reliable JSON: tool whose input_schema = output schema, force it, then validate+repair
- Prefill assistant with { to force JSON / skip preamble
- Cut hallucination: allow 'not found', answer only from docs, quote/cite
- Durable rules in system prompt (also helps caching)
- Evaluate prompts with an eval set; change one variable at a time

D4 · Context & Reliability (~15%)

- Context engineering = curate the whole token set; window is scarce ('context rot')
- Prompt caching: static prefix FIRST, byte-identical, variable query LAST
- Cache writes slightly pricier; reads much cheaper+faster (wins in agent loops)
- Long sessions: compact/summarize old turns + external memory
- Reliability: validate+retry, exponential backoff, self-consistency, evals
- Cost/latency: Batches API, prompt caching, model routing, streaming
- RAG quality is bounded by retrieval quality +

HOW TO STUDY (evidence-based)

- Retrieve, don't re-read: testing beats rereading (~50% more retained)
- Space reviews over days (flashcards 'due' pile each session)
- Interleave domains (quiz mixed mode, full exams)
- Explain it back (Feynman) using each note's teach-it-back list
- Simulate: 2 timed 60-Q exams; remediate every miss
- Readiness: >=80% per-domain quiz, >=45/60 on a fresh exam